

## Experiment No. 1 Experiment Name Implementation of Fundamental Image Processing Operations using Python and OpenCV

**Aim** To develop a comprehensive understanding of fundamental image processing techniques by implementing image acquisition, storage, color space conversion, geometric transformations, and image enhancement operations using Python and OpenCV, thereby establishing the foundation for advanced computer vision applications.

**Purpose** Image processing forms the core of modern computer vision systems used in domains such as autonomous driving, healthcare, surveillance, robotics, and industrial automation. Before performing advanced tasks like feature extraction, object detection, or image classification, it is essential to understand how digital images are represented, manipulated, and transformed. In this experiment, students will implement fundamental image processing operations using Python and OpenCV. They will learn to read, write, convert, resize, and manipulate digital images while understanding how these operations influence image quality and prepare images for subsequent computer vision tasks.

**Prerequisites** • Basic knowledge of Python programming • Familiarity with digital image representation (pixels, resolution, color channels) • Python libraries: OpenCV (cv2), NumPy, and Matplotlib • Jupyter Notebook, Google Colab, or Visual Studio Code

### Steps

1. Install and import the required libraries, including OpenCV, NumPy, and Matplotlib.
2. Load a color image from the local system and display it using both OpenCV and Matplotlib.
3. Examine the image properties such as dimensions, resolution, number of channels, and data type.
4. Save the image in different file formats (e.g., JPEG and PNG) and compare the outputs.
5. Convert the image from RGB/BGR to Grayscale, HSV, and LAB color spaces.
6. Perform basic geometric transformations such as resizing, rotation, and horizontal/vertical flipping.
7. Generate the complement (negative) of the image by inverting pixel intensity values.
8. Crop a Region of Interest (ROI) from the image and analyze its characteristics.
9. Display the original and processed images for comparison and observe the effects of each operation.
10. Document the observations and discuss the significance of these preprocessing techniques in computer vision applications.

```
# Step 1: Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# Optional: for displaying images in notebook
%matplotlib inline
```

```
# Step 2: Read image from local system
img_path = "sample.jpg" # Replace with your image file name

img_bgr = cv2.imread(img_path)

if img_bgr is None:
    raise FileNotFoundError(f"Image not found: {img_path}")

# Convert BGR to RGB for Matplotlib display
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Display using Matplotlib
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Image using Matplotlib")
plt.axis("off")

# Display using OpenCV
# This works in local Python script or terminal environment
cv2.imshow("Image using OpenCV", img_bgr)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Image using Matplotlib



```
# Step 3: Image properties
height, width, channels = img_bgr.shape
print("Image shape:", img_bgr.shape)
```

```

print("Height:", height)
print("Width:", width)
print("Channels:", channels)
print("Data type:", img_bgr.dtype)
print("Resolution:", width, "x", height)

Image shape: (477, 500, 3)
Height: 477
Width: 500
Channels: 3
Data type: uint8
Resolution: 500 x 477

# Step 4: Save in JPEG and PNG
cv2.imwrite("saved_image.jpg", img_bgr)
cv2.imwrite("saved_image.png", img_bgr)

# Check file sizes
for file_name in ["saved_image.jpg", "saved_image.png"]:
    size = os.path.getsize(file_name)
    print(f"{file_name}: {size} bytes")

saved_image.jpg: 52141 bytes
saved_image.png: 219091 bytes

# Step 5: Convert to different color spaces
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)

# Display all versions
plt.figure(figsize=(16, 6))

plt.subplot(1, 4, 1)
plt.imshow(img_rgb)
plt.title("Original RGB")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")

```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Step 6: Resize, rotate, and flip
resized = cv2.resize(img_bgr, (width // 2, height // 2))
rotated = cv2.rotate(img_bgr, cv2.ROTATE_90_CLOCKWISE)
flipped_horizontal = cv2.flip(img_bgr, 1)
flipped_vertical = cv2.flip(img_bgr, 0)

# Display
plt.figure(figsize=(18, 10))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(flipped_horizontal, cv2.COLOR_BGR2RGB))
plt.title("Flipped Horizontal")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(flipped_vertical, cv2.COLOR_BGR2RGB))
plt.title("Flipped Vertical")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



```
# Step 7: Negative image
negative = 255 - img_bgr

plt.figure(figsize=(8, 6))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original



Negative Image



```
# Step 8: Crop ROI
roi = img_bgr[100:300, 200:500] # y1:y2, x1:x2

print("ROI shape:", roi.shape)

plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Cropped ROI")
plt.axis("off")

plt.tight_layout()
plt.show()

ROI shape: (200, 300, 3)
```



*# Step 9: Comparison of original and processed outputs*  
`plt.figure(figsize=(18, 12))`

```
plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
```

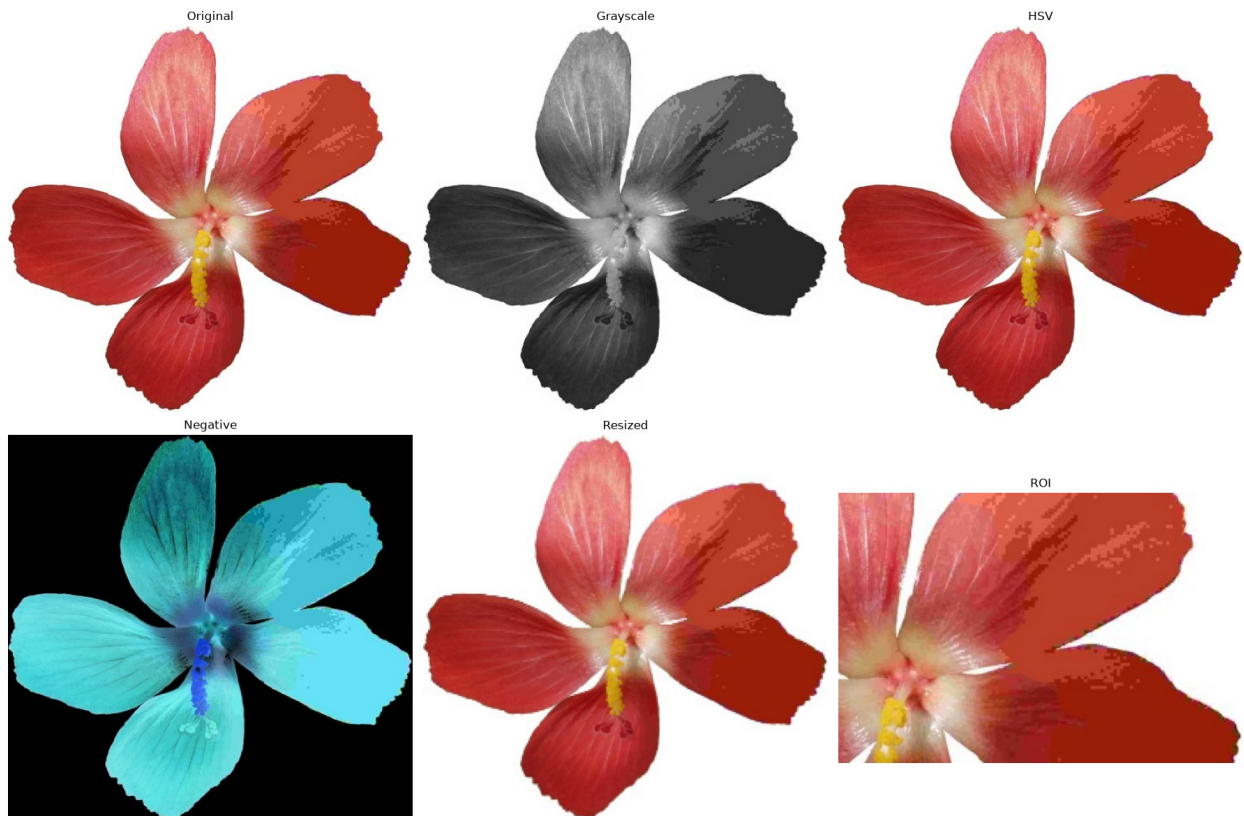
```
plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("ROI")
```

```
plt.axis("off")  
  
plt.tight_layout()  
plt.show()
```



```
# Step 10: Observation notes (optional text output)  
print("Observations:")  
print("1. Color space conversion changes how the image is represented,  
useful for segmentation and enhancement.")  
print("2. Resizing changes image dimensions, which affects processing  
speed and quality.")  
print("3. Rotation and flipping help in orientation correction and  
augmentation.")  
print("4. Negative image highlights intensity inversion for contrast  
analysis.")  
print("5. ROI cropping extracts relevant regions for focused  
processing.")  
print("6. These preprocessing steps are essential before feature  
extraction, object detection, and classification.")
```

Observations:

1. Color space conversion changes how the image is represented, useful for segmentation and enhancement.
2. Resizing changes image dimensions, which affects processing speed

and quality.

3. Rotation and flipping help in orientation correction and augmentation.

4. Negative image highlights intensity inversion for contrast analysis.

5. ROI cropping extracts relevant regions for focused processing.

6. These preprocessing steps are essential before feature extraction, object detection, and classification.

## Questions

1. What is a digital image? Differentiate between grayscale and color images.
2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.
3. What is the purpose of converting an image to grayscale before further processing?
4. Explain the concept of image complement (negative image) and mention its practical applications.
5. Differentiate between image resizing, cropping, and scaling.
6. What is a Region of Interest (ROI), and why is it important in computer vision?
7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?
8. Explain how image resolution and pixel intensity influence image quality.
9. Mention five real-world applications where basic image preprocessing is an essential step.
10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Here are clear, exam-ready answers to all 10 questions:

## 1. What is a digital image? Differentiate between grayscale and color images.

A **digital image** is a two-dimensional representation of a scene or object made up of small elements called **pixels (picture elements)**. Each pixel contains numerical information representing the intensity or color at that location.

| Grayscale Image                                                                    | Color Image                                                           |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Contains only shades of gray.                                                      | Contains multiple color channels.                                     |
| Usually has <b>one channel</b> .                                                   | Usually has <b>three channels</b> .                                   |
| Pixel values commonly range from <b>0 (black) to 255 (white)</b> for 8-bit images. | In RGB, each channel commonly ranges from 0–255.                      |
| Requires less storage and computation.                                             | Requires more storage and computation.                                |
| Used for tasks such as edge detection and thresholding.                            | Used when color information is important, such as object recognition. |

## 2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

- **RGB (Red, Green, Blue):** Represents an image using three channels—red, green, and blue. It is commonly used for displaying images on screens.
- **BGR (Blue, Green, Red):** Similar to RGB, but the channel order is blue, green, and red. **OpenCV commonly uses BGR as its default color representation.**
- **HSV (Hue, Saturation, Value):**
  - **Hue:** Represents the type of color.
  - **Saturation:** Represents color intensity/purity.
  - **Value:** Represents brightness.

HSV is useful for **color detection and segmentation** because color information is separated from brightness.

- **LAB:** Consists of **L (lightness)**, **A (green-to-red)**, and **B (blue-to-yellow)** components. It is designed to be closer to human visual perception and is useful for **color correction, image enhancement, and color comparison.**

## 3. What is the purpose of converting an image to grayscale before further processing?

Converting an image to grayscale reduces a color image from three channels to **one intensity channel**. This:

- Reduces computational complexity.
- Requires less memory.
- Simplifies mathematical operations.
- Makes algorithms such as **edge detection, thresholding, contour detection, and feature extraction** easier to apply.
- Removes color information when color is not relevant to the task.

For example, edge detection often depends mainly on changes in brightness rather than color.

## 4. Explain the concept of image complement (negative image) and mention its practical applications.

An **image complement**, or **negative image**, reverses the intensity values of an image.

For an 8-bit grayscale image:

**Negative intensity = 255 – Original intensity**

Thus, black becomes white, white becomes black, and intermediate gray values are inverted.

**Applications include:**

- Enhancing details in dark regions.

- Medical image analysis.
- Viewing X-ray and photographic images.
- Improving visibility of certain features.
- Historical film and document processing.

## 5. Differentiate between image resizing, cropping, and scaling.

- **Resizing:** Changes the overall dimensions of an image, such as changing 1920×1080 to 640×360.
- **Cropping:** Removes unwanted portions of an image and keeps only a selected region.
- **Scaling:** Changes the size of an image or its objects by a particular factor, such as scaling an image by 0.5× or 2×.

In practice, resizing/scaling generally involves interpolation, while cropping simply selects and retains a portion of the original pixels.

## 6. What is a Region of Interest (ROI), and why is it important in computer vision?

A **Region of Interest (ROI)** is a specific portion of an image selected for analysis.

For example, in a traffic-monitoring system, the ROI could be the area containing vehicles while ignoring the sky and buildings.

### Importance of ROI:

- Reduces the amount of data to process.
- Improves processing speed.
- Focuses algorithms on relevant information.
- Can improve detection and recognition accuracy.
- Reduces interference from irrelevant background regions.

## 7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

**OpenCV (Open Source Computer Vision Library)** is widely preferred because it is specifically designed for computer vision and provides a large collection of optimized functions.

Major advantages include:

- Open-source and freely available.
- Supports **Python, C++, Java**, and other languages.
- Provides functions for image processing, object detection, video processing, and feature extraction.
- Efficient and fast, with optimized implementations.
- Supports real-time computer vision applications.
- Provides easy image and video input/output.
- Integrates well with **NumPy and deep learning frameworks**.
- Has extensive documentation and community support.

## 8. Explain how image resolution and pixel intensity influence image quality.

**Image resolution** refers to the number of pixels used to represent an image. Higher resolution generally provides more detail and sharper images because more spatial information is available. However, it also increases storage and computational requirements.

**Pixel intensity** represents the brightness of a pixel. In a typical 8-bit grayscale image:

- **0 = black**
- **255 = white**
- Values between 0 and 255 represent different shades of gray.

Proper intensity distribution improves **contrast and visibility**. Poor intensity distribution, such as an image that is too dark or too bright, can hide important details.

## 9. Mention five real-world applications where basic image preprocessing is an essential step.

Five examples are:

1. **Medical imaging** – enhancing X-rays, CT scans, and MRI images.
2. **Face recognition** – resizing, grayscale conversion, and normalization before recognition.
3. **OCR (Optical Character Recognition)** – removing noise and improving document contrast.
4. **Autonomous vehicles** – preprocessing camera images for road, vehicle, and pedestrian detection.
5. **Satellite and remote sensing** – enhancing and correcting images for land-use and environmental analysis.

Other applications include industrial inspection, security surveillance, biometric systems, and traffic monitoring.

## 10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Image preprocessing improves performance by making input images **cleaner, more consistent, and easier to analyze**.

Important techniques include:

- **Resizing:** Ensures all input images have the required dimensions.
- **Grayscale conversion:** Reduces unnecessary color information for tasks where color is not important.
- **Noise reduction:** Removes unwanted variations that can interfere with feature detection.
- **Contrast enhancement:** Makes important structures and edges more visible.
- **Normalization:** Places pixel values into a consistent range, helping models train more effectively.
- **Cropping/ROI selection:** Removes irrelevant background information.

- **Data augmentation:** Creates variations such as rotations, flips, and changes in scale, which can improve model generalization.

Overall, preprocessing can lead to **better feature extraction, faster computation, more stable training, improved accuracy, and better generalization** of computer vision and deep learning models.